Recognising cellular automata

Simon Frost

Table of contents

1 Overview 1

2 A map given as a black box 2

3 Maps that are not cellular automata 2

4 Large groups 3

1 Overview

The Curtis–Hedlund–Lyndon theorem says which maps of configurations are cellular automata: those that commute with the shifts and are uniform, that is, whose value at a cell depends on a finite part of the configuration around it. The theorem holds for alphabets in any category with products (Castillo-Ramirez et al. 2026). Over a finite group every map is uniform, with the whole group as its memory set, so commuting with the shifts is the whole test.

This vignette uses the theorem in practice. It tests whether a map, given only as a function, is a cellular automaton. When it is, the map's local rule is recovered and its memory set shrunk to the cells it depends on. It also shows which familiar operations fail the test, and why.

```
ENV["GKSwstype"] = "100"
using CategoricalCellularAutomata
using CellularAutomataCore: CellularAutomataCore, Wolfram, Life, Torus, Grid,
CAProblem,
                            solve
using Random
```

2 A map given as a black box

Here is a smoothing filter on a ring of 12 cells, written with array operations and no mention of neighbourhoods. Each cell takes the majority of itself and the two cells on either side, a window of five:

```
Z = CyclicGroup(12)
B = FinAlphabet(0:1)
function smooth(x)
    u = parent(x)
    total = sum(circshift(u, k) for k in -2:2)
    return Configuration(Z, Int.(total .>= 3))
end
```

There are `2^12` configurations, few enough to try them all, and the map commutes with every shift:

```
is_equivariant(smooth, Z, B)
```

```
true
```

So it is a cellular automaton. `cellular_automaton` turns it into one, with the whole group as its memory set and the map read at the identity cell as its local rule. `minimal_neighbourhood` then keeps the cells the rule depends on:

```
τ = cellular_automaton(smooth, Z; dom=B)
m = minimal_neighbourhood(τ)
(length(memory_set(τ)), sort(memory_set(m)))
```

```
(12, [0, 1, 2, 10, 11])
```

The recovered memory set is the window `{-2, …, 2}` (with `-1` and `-2` written as 11 and 10), and the automaton equals the original map:

```
x = Configuration(Z, rand(Xoshiro(1), 0:1, 12))
(m(x) == smooth(x), m == τ)
```

```
(true, true)
```

3 Maps that are not cellular automata

A rule run on a bounded grid is not a cellular automaton over the ring. Here is one step of elementary rule 90 on 12 cells with the edges held at 0, using CellularAutomataCore. The cells at the edges see a different neighbourhood from the others, so the map does not commute with the shifts. The same rule on a ring does.

```
step_on(space) = x -> Configuration(Z, solve(CAProblem(Wolfram(90),
Int8.(parent(x)), 1;
                                                    space)).u[end])
B8 = FinAlphabet(Int8[0, 1])
(bounded=is_equivariant(step_on(Grid(12)), Z, B8),
 ring=is_equivariant(step_on(Torus(12)), Z, B8))
```

```
(bounded = false, ring = true)
```

On the ring, the recovered rule has the memory set of rule 90, the two neighbours of each cell:

```
sort(memory_set(minimal_neighbourhood(cellular_automaton(step_on(Torus(12)), Z;
dom=B8))))
```

```
2-element Vector{Int64}:
  1
 11
```

A map can also fail by treating elements differently through the group operation. On the symmetric group S3, reading the cells `a * g` for a fixed element `a` is a cellular automaton, but reading the inverse g^{-1} is not:

```
perms = [[1, 2, 3], [2, 1, 3], [1, 3, 2], [3, 2, 1], [2, 3, 1], [3, 1, 2]]
S3 = FiniteGroup(perms, (a, b) -> a[b])
left(x) = Configuration(g -> x[mul(S3, 5, g)] & x[g], S3)
inverse_read(x) = Configuration(g -> x[inverse(S3, g)] & x[g], S3)
(left=is_equivariant(left, S3, B), inverse=is_equivariant(inverse_read, S3, B))
```

```
(left = true, inverse = false)
```

4 Large groups

On a 6×6 torus there are `2^36` configurations, too many to try, and `is_equivariant` refuses to guess. Given a random number generator, it tests a sample instead. A sample can show that a map is not a cellular automaton, but cannot prove that it is.

```
T = TorusGroup(6, 6)
life_step(x) = Configuration(T, solve(CAProblem(Life(), Int8.(parent(x)),
1)).u[end])
try
    is_equivariant(life_step, T, B8)
catch err
    first(sprint(showerror, err), 70) * "..."
end
```

```
"ArgumentError: there are 68719476736 configurations, too many to enume..."
```

```
is_equivariant(life_step, T, B8; rng=Xoshiro(2), samples=50)
```

```
true
```

Recovering the local rule from the map would mean enumerating patterns over all 36 cells, which is also out of reach. Starting instead from the core rule, whose memory set is the cell and its eight neighbours, `minimal_neighbourhood` confirms that Life depends on all nine:

```
life = CellularAutomaton(Life(), Torus(6, 6); alphabet=B8)
length(memory_set(minimal_neighbourhood(life)))
```

```
9
```

The checks in this vignette work on small groups and small memory sets, which is where questions about what a rule is are usually asked. The automata they produce run on CellularAutomataCore at any size.

Castillo-Ramirez, Alonso, Alejandro Vazquez-Aceves, and Angel Zaldivar-Corichi. 2026. *A Categorical Framework for Cellular Automata*. https://arxiv.org/abs/2602.04049.